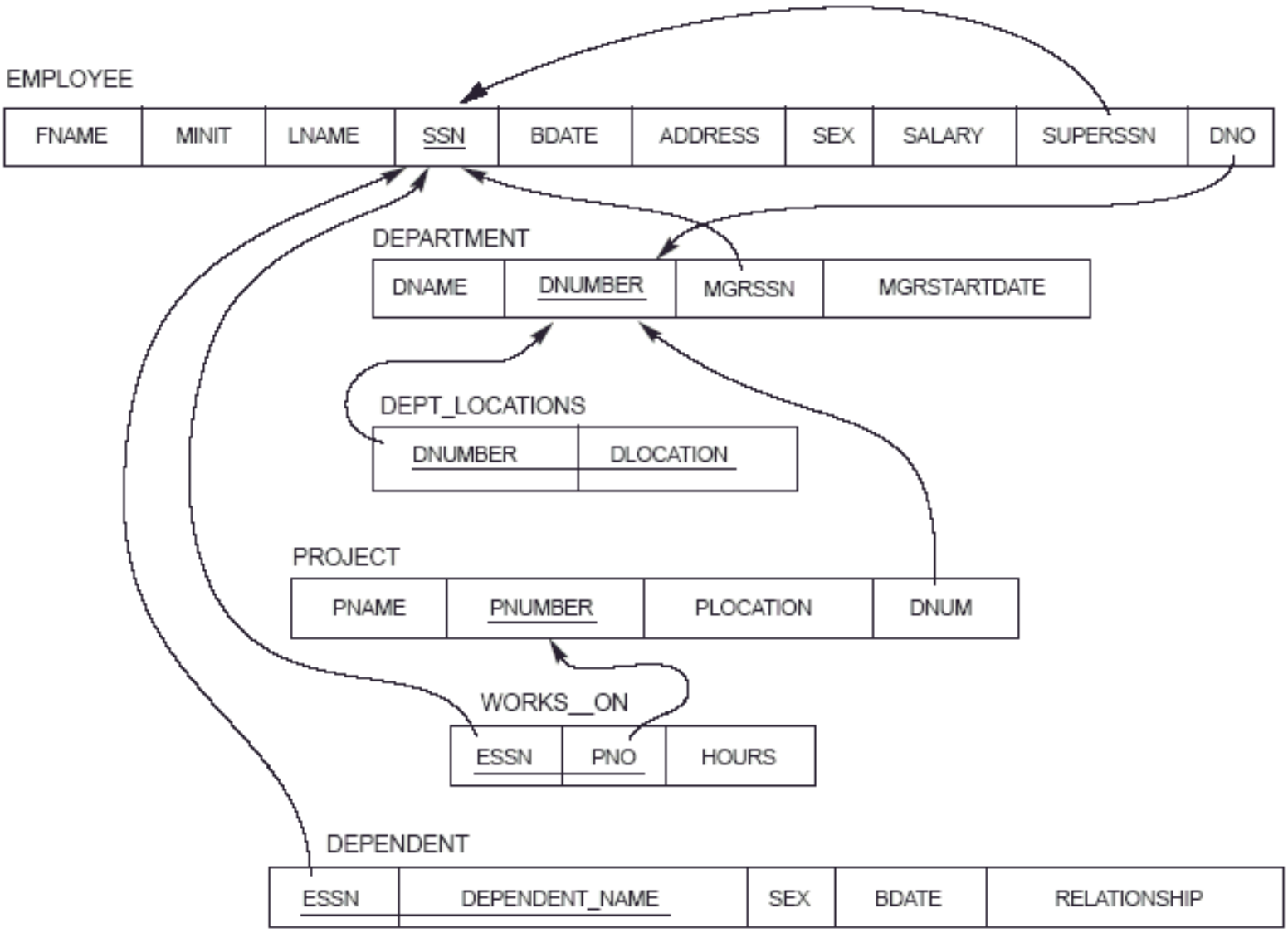
# SQL – The Relational Database Standard

**Figure 7.7** Referential integrity constraints displayed on the COMPANY relational database schema diagram.



# SQL

The different **DDL commands** are as follows.

- CREATE TABLE
- ALTER TABLE …….ADD
- ALTER TABLE…..MODIFY
- DROP TABLE

# SQL

## SQL DML

- Interactive SQL has many powerful queries. An important set of such queries is given below
  - INSERT INTO
  - UPDATE-SET-WHERE
  - DELETE-FROM
  - SELECT-FROM
  - SELECT-FROM-WHERE
  - SELECT-FROM-GROUP BY
  - SELECT-FROM-ORDER BY

# SQL

- SELECT <attribute and function list>
  FROM <table list>
  [WHERE <condition>]
  [GROUP BY <grouping attributes(s)>]
  [HAVING <group condition>]
  [ORDER BY <attribute list>]

# SQL

- Retrieve the name and address of all employees who work for the ‘Research’ department.

```
SELECT    FNAME,LNAME,ADDRESS
FROM      EMPLOYEE,DEPARTMENT
WHERE     DNAME=‘Research’ AND
          DNUMBER=DNO;
```

# SQL

- **Eliminating duplicates**
  - SQL does not automatically eliminate the duplicate tuples in the results of queries.
  - To avoid this, the keyword SELECT should be followed by the keyword **DISTINCT**. Then, the resulting relation only contain one copy of any duplicated tuples.

# SQL

- Retrieve the salary of every employee and all distinct salary values

```
SELECT ALL      SALARY
FROM            EMPLOYEE
```

```
SELECT DISTINCT SALARY
FROM            EMPLOYEE
```

# SQL

- **Comparison of strings**
  - Compare two strings on the basis of a simple pattern match

    e.g. s LIKE p (s is a string, p is a pattern)

    s not like p
  - Special characters:
    - %: match any sequence of 0 or more characters in s
    - _: match any one character in s

# SQL

- Retrieve all employees whose address is in Houston, Texas

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE ADDRESS LIKE '%Houston,TX%';
```

Find all employees who were born during the 1950's

# SQL

- **Arithmetic in Queries (+, -, *, / )**

Show the result salaries if every employee working on the ‘ProductX’ project is given a 10 percent raise

```
SELECT    FNAME,LNAME,1.1*SALARY
FROM      EMPLOYEE, WORKS_ON,
          PROJECT
WHERE     SSN=ESSN AND
          PNO=PNUMBER AND
          PNAME=‘ProductX’;
```

# SQL

- **Comparison operator – BETWEEN**

  The BETWEEN … AND operator selects a range of data between two values. These values can be numbers, text, or dates

Retrieve all employees in department 5 whose salary is between $30,000 and $40,000

```
SELECT    *
FROM      EMPLOYEE
WHERE     (SALARY BETWEEN 30000 AND
          40000) AND DNO=5
```

# SQL

- **Ordering the output – ORDER BY**
  - The ORDER BY keyword is used to sort the result
  - The result to be presented in sorted order.
  - Default order is ascending, you can define as DESC (descending)

Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, first name.

# SQL

```
SELECT    DNAME, LNAME, FNAME, PNAME
FROM      DEPARTMENT, EMPLOYEE,
          WORKS_ON, PROJECT
WHERE     DNUMBER=DNO AND SSN=ESSN AND
          PNO=PNUMBER
ORDER BY  DNAME, LNAME, FNAME

ORDER BY  DNAME DESC, LNAME ASC,
          FNAME ASC
```

# SQL

- **Nested Queries and Set Comparisons**

  IN….

  The IN operator may be used if you know the exact value you want to return for at least one of the columns.

- Make a list of all project numbers for projects that involve an employee whose last name is ‘Smith’, either as a worker or as a manager of the department that controls the project

# SQL

```
SELECT      DISTINCT PNUMBER
FROM        PROJECT
WHERE       PNUMBER IN
        ( SELECT    PNUMBER
          FROM      PROJECT, DEPARTMENT, EMPLOYEE
          WHERE     DNUM=DNUMBER AND MGRSSN=SSN
                    AND LNAME='Smith')
        OR  PNUMBER IN
        ( SELECT    PNO
          FROM      WORKS_ON, EMPLOYEE
          WHERE     ESSN=SSN AND LNAME='Smith');
```

# SQL

- **Aggregate Functions and Grouping**

  Built-in functions: **COUNT**, **SUM**, **MAX, MIN**, **AVG**
  **COUNT** –Returns the number of tuples or values

  These functions can be used in the **SELECT – clause or in a HAVING-clause.**

# SQL

- **GROUP BY….**
  - GROUP BY… was added to SQL because aggregate functions (like SUM) return the aggregate of all column values every time they are called, and without the GROUP BY function it was impossible to find the sum for each individual group of column values.

# SQL

- Example

| Company | Amount |
|---|---|
| MAHE | 5500 |
| MSIS | 1200 |
| MAHE | 7300 |

**SELECT Company, SUM(Amount) FROM Sales**

| Company | Amount |
|---|---|
| MAHE | 14000 |
| MSIS | 14000 |
| MAHE | 14000 |

# SQL

- The previous code is invalid because the column returned is not part of an aggregate. A GROUP BY clause will solve this problem:

**SELECT Company,SUM(Amount) FROM Sales GROUP BY Company**

| Company | Amount |
|---|---|
| MAHE | 12800 |
| MSIS | 1200 |

# SQL

- **HAVING……**

  HAVING… was added to SQL because the WHERE keyword could not be used against aggregate functions (like SUM), and without HAVING… it would be impossible to test for result conditions

# SQL

- SELECT Company,SUM(Amount)
  FROM Sales
  GROUP BY Company
  HAVING SUM(Amount)>10000

| **Company** | **Amount** |
|---|---|
| MAHE | 12800 |

# SQL

- **Views (Virtual Tables ) in SQL**

  A view in SQL terminology is a single table that is derived from other tables. These other tables could be base tables or previously defined views. A view is considered a virtual table, in contrast to base tables whose tuples are actually stored in the database.

# SQL

- CREATE VIEW WORKS_ON1
  AS SELECT FNAME,LNAME,PNAME,HOURS
  FROM EMPLOYEE, PROJECT, WORKS_ON
  WHERE SSN-ESSN AND PNO=PNUMBER

# SQL

- CREATE VIEW DEPAT_INFO(DEPT_NAME,
  NO_OF_EMPS, TOTAL_SAL)
  AS SELECT DNAME,COUNT(*),SUM(SALARY)
  FROM DEPARTMENT, EMPLOYEE
  WHERE DNUMBER=DNO
  GROUP BY DNAME

# SQL

- **Stored Procedures**

- A stored procedure is nothing more than prepared SQL code that you save so you can reuse the code over and over again.
- Then just call the stored procedure to execute the SQL code that you saved as part of the stored procedure.

# SQL

- Syntax:

```
CREATE PROCEDURE procedure_name
[ @parameter_name data_type] [= default] OUTPUT][,...,n]

AS

  T-SQL_statement(s)
```

# SQL

Example:

```
CREATE PROC Departments_Members
AS
  SELECT Dep_Name, COUNT(Emp_ID) NumberOfMember
  FROM Departments D, Employees E
  WHERE D.Dep_ID = E.Dep_ID
  GROUP BY Dep_Name
```

Run Procedure

```
  Execute Departments_Members
```

# SQL

```
CREATE PROC Department_Members @DeptName varchar(50)
AS
  SELECT Dep_Name, COUNT(Emp_ID) NumberOfMember
  FROM Departments D, Employees E
  WHERE D.Dep_ID = E.Dep_ID and Dep_Name = @DeptName
  GROUP BY Dep_Name
```

Run Procedure

```
  Execute Department_Members  'Accounting'
```

# SQL

- Update a procedure

```
ALTER PROC[EDURE] procedure_name
      [ @parameter_name data_type]
            [= default] [OUTPUT]
      [,...,n]
AS
    t-sql_statement(s)
```

# SQL

- Delete a procedure
  - DROP PROCEDURE  procedure_name